Chip điều khiển PID cho drone

Giải thích thiết kế cho cả nhóm — dễ hiểu, không cần biết trước về thiết kế vi mạch

Nhóm FPOLY UAV · Cuộc thi Thiết kế vi mạch cho Đô thị thông minh lần 3 · 24/09/2026

> **Đọc nhanh:** nhóm đã có drone bay được với thuật toán PID chạy trên STM32. Tài liệu này giải thích cách biến thuật toán đó thành một con chip riêng: đã làm được gì (mô hình số nguyên, mạch Verilog, bộ kiểm thử đã PASS), vì sao chọn như vậy, và còn phải làm gì tiếp. Mục 6–8 khớp với bộ slide Bán kết (waveform, đóng gói SPI, bảng so sánh). Mục 11 có bảng giải nghĩa các thuật ngữ.

1. Thiết kế chip khác gì lập trình vi điều khiển?

Vi điều khiển (MCU) như STM32 giống một đầu bếp đa năng. Bên trong có một bộ xử lý (CPU) biết làm mọi việc. Ta đưa nó "công thức" là code C (`Flight.cpp`), nó đọc từng dòng rồi làm theo. Đổi code là nó làm việc khác.

Chip tự thiết kế giống một cái máy chỉ làm đúng một món. Không có CPU, không có code: mạch điện được nối cứng để làm đúng việc tính PID. Vì không phải "đọc công thức" nên chip nhỏ hơn, tốn ít điện hơn và nhanh hơn; đổi lại, làm xong là không sửa được.

Mục tiêu của nhóm: biến bộ điều khiển PID + bộ trộn động cơ đang chạy trên STM32 thành một mạch điện chuyên dụng, cho kết quả giống hệt bản đã bay thử.

2. Quy trình thiết kế chip và tiến độ



Hình 1. Bảy bước làm một con chip. Bốn bước đầu đã xong trên máy của nhóm; ba bước sau cần công cụ Cadence.

Bước	Làm gì	Công cụ	Trạng thái
1. Thuật toán	Viết và bay thử thuật toán bằng phần mềm	STM32, `Flight.cpp`	Xong — drone bay được
2. Mô hình fixed-point	Chuyển số thập phân sang số nguyên để mạch tính được	Python	Xong
3. RTL	"Vẽ" mạch điện bằng ngôn ngữ Verilog	Verilog	Xong, biên dịch không lỗi
4. Mô phỏng	Chạy thử mạch trên máy tính, so với đáp án	Icarus Verilog	Xong — PASS 16.000/16.000 mẫu
5. Tổng hợp	Đổi Verilog thành hàng nghìn cổng logic thật	Cadence Genus	Đang làm — đã có script `syn/syn_genus.tcl`, chưa có số đo
6. Layout	Xếp cổng lên mặt silicon và đi dây	Cadence Innovus	Chưa làm
7. Kiểm tra cuối	Kiểm tra tốc độ, luật vẽ; xuất file gửi nhà máy	Tempus, Pegasus	Chưa làm

3. Bước 2 — Mô hình fixed-point

3.1. Vì sao phải đổi sang số nguyên?

Code STM32 dùng **số thực (float)**, ví dụ góc = 12,3456°. Mạch tính số thực rất to và phức tạp, nên chip nhỏ gọn chỉ làm với **số nguyên**. Mẹo giống như ghi tiền: thay vì 12,35 nghìn đồng ta ghi 12350 đồng — vẫn chính xác mà chỉ còn số nguyên.

Mạch cũng vậy: nhân mọi giá trị với một số cố định rồi lưu thành số nguyên. Ví dụ với `fe = 10`, góc được nhân với 2^{10} = 1024: góc 12,3456° lưu thành 12642, và bước nhỏ nhất chip phân biệt được là 1/1024 ≈ 0,001°.

Câu hỏi then chốt là dùng bao nhiêu bit (bit = chữ số nhị phân). Nhiều bit thì chính xác nhưng mạch to và tốn điện; ít bit thì mạch nhỏ nhưng tính lệch so với STM32.

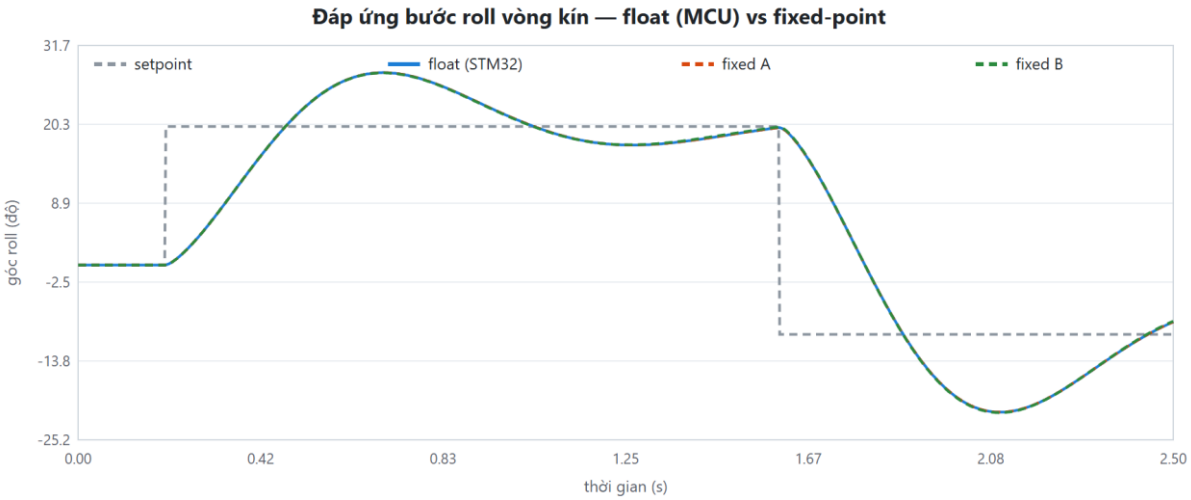
3.2. Cách chọn số bit

Mô hình Python (`ic/model/`) chạy **song song** hai bản PID: bản số thực chép y hệt `Flight.cpp`, và bản số nguyên giống hệt mạch. Cả hai nhận cùng dữ liệu bay từ 7 kịch bản (16.000 chu kỳ): lơ lửng có gió, đổi góc đột ngột, xoay yaw, gạt cần ngẫu nhiên, drone bị kẹt gây bão hoà, đầu vào ngẫu nhiên toàn dải, arm/disarm liên tục. Mô hình thử lần lượt từng mức bit để tìm **mức ít bit nhất mà kết quả vẫn gần như giống hệt**.

3.3. Kết quả: hai lựa chọn cho nhóm

Cấu hình	Lệnh motor lệch so với STM32	Góc bay lệch (vòng kín)	Phép nhân lớn nhất	Nhận xét
A — khớp MCU	tối đa 1 µs, ở 0,71% chu kỳ	0,013°	40 bit	Dễ chứng minh "chip = thuật toán đã bay thử"
B — tiết kiệm	tối đa 4 µs, ở 14% chu kỳ	0,11°	32 bit	Nhỏ hơn, lệch vẫn dưới nhiễu cảm biến (0,3°)
C — tối giản	tối đa 18 µs, ở 61% chu kỳ	1,24°	28 bit	Bắt đầu thấy khác biệt — không dùng

Cảm biến IMU vốn có nhiễu khoảng 0,3°, nên với cấu hình B drone không thể "cảm thấy" khác biệt. Mạch hiện tại dùng **cấu hình A**; đổi sang B chỉ cần chạy lại một lệnh, không phải sửa mạch.



Hình 2. Đáp ứng khi đổi góc roll (0° → 20° → −10°) trên mô hình drone đơn giản: bản số thực (STM32) và hai bản số nguyên gần như trùng khít. Độ vọt lố là do mô hình drone giả lập, không phải drone thật.

3.4. Mẹo để chip không cần phép chia

Phép chia trong mạch rất tốn diện tích. Code STM32 có chia cho dt (thời gian giữa hai lần tính). Vì chip luôn tính đều 400 lần/giây, dt luôn bằng 2,5 ms, nên phần chia được **tính sẵn vào hệ số** từ trước. Chip chỉ còn cộng, nhân và dịch bit.

Trên STM32 (số thực)	Trên chip (số nguyên)
dt đo từ đồng hồ, thay đổi từng lần	dt cố định 2,5 ms (400 Hz)
Tích phân: I = I + e·dt, giới hạn ±100	A = A + e, giới hạn ±100/dt (chỉ cộng)
Đạo hàm: (e − e_trước) / dt	ΔE = e − e_trước (không chia)
u = kp·e + ki·I + kd·D, rồi nhân 3	u' = KP'·e + KI'·A + KD'·ΔE với KP' = 3kp, KI' = 3ki·dt, KD' = 3kd/dt

3.5. Những phát hiện đáng đưa vào báo cáo thi

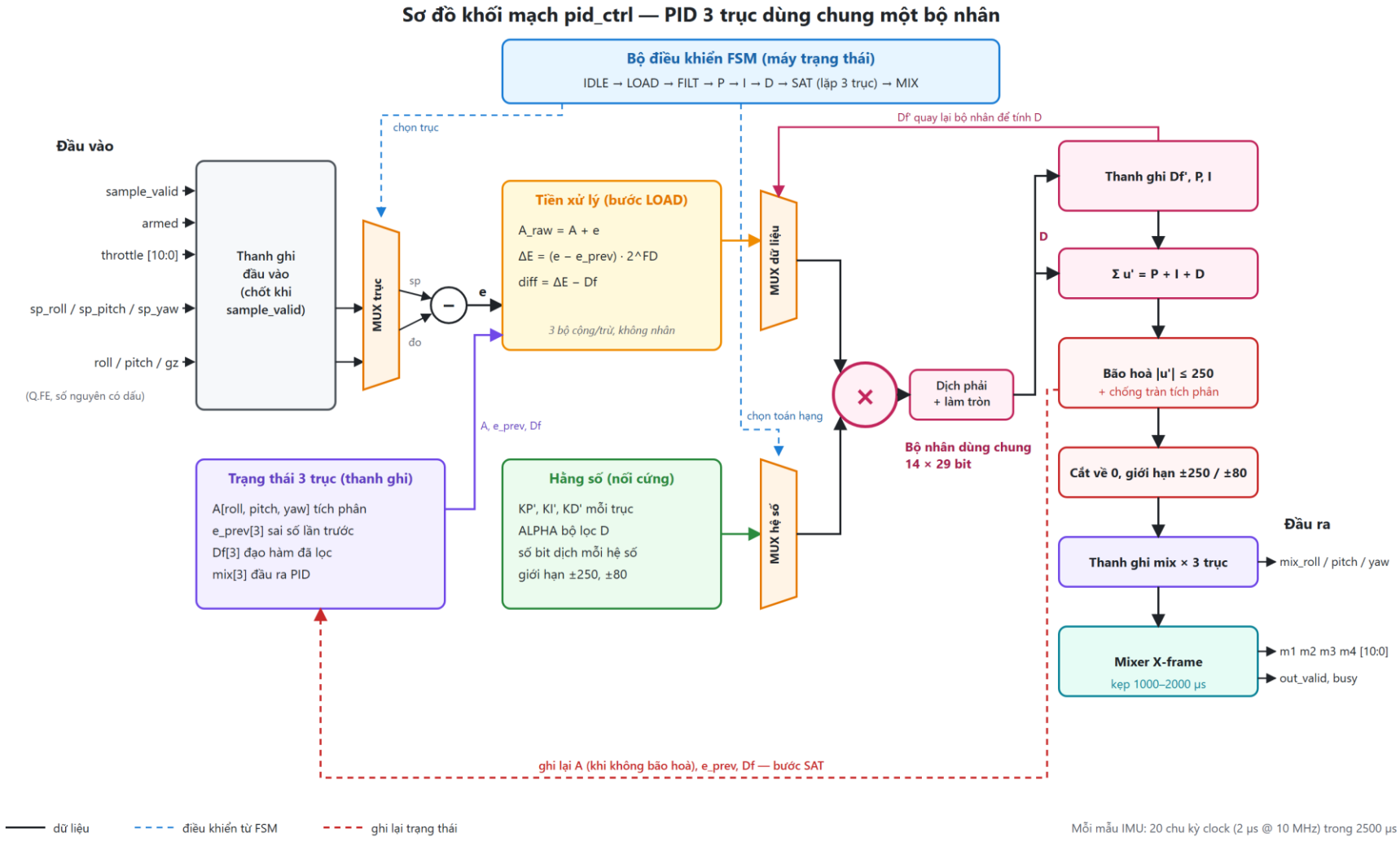
- **Thành phần D quyết định độ chính xác đầu vào.** KD' = 36 nên sai số làm tròn của góc bị nhân lên 36 lần; đây là lý do cấu hình A cần tới 10 bit phần thập phân cho góc.
- **Hệ số lọc alpha = 0,2 tốn 13 bit** vì không biểu diễn gọn được ở hệ nhị phân. Nếu đổi thành 13/64 = 0,203 trên cả STM32 lẫn chip (bộ lọc gần như y hệt), chỉ cần 6 bit.
- **Bắt buộc làm tròn khi dịch bit.** Nếu cắt bỏ phần lẻ, cấu hình A lệch 2 µs và không đạt tiêu chí; làm tròn chỉ tốn thêm một bộ cộng.

4. Bước 3 — Kiến trúc mạch (RTL)

4.1. Verilog không phải chương trình

Verilog trông giống code nhưng **không chạy từng dòng**. Mỗi dòng mô tả một **linh kiện** và cách nối dây; công cụ sẽ dựng mạch thật từ mô tả đó. Mạch PID gồm các loại linh kiện chính: **thanh ghi** (ô nhớ nhỏ giữ một số), **bộ cộng, bộ nhân**, **bộ chọn (MUX)** chọn một trong nhiều đường dữ liệu, và **máy trạng thái (FSM)** điều phối mỗi nhịp clock làm việc gì.

4.2. Sơ đồ khối

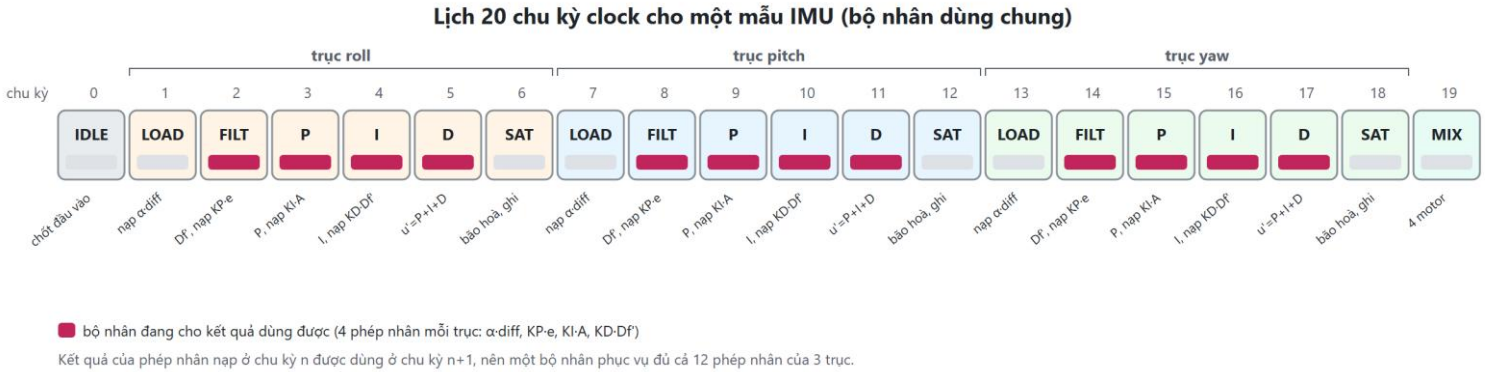


Hình 3. Sơ đồ khối mạch `pid_ctrl`. Nét liền là dữ liệu, nét đứt xanh là điều khiển từ FSM, nét đứt đỏ là ghi lại trạng thái sau mỗi lần tính.

Khối	Nhiệm vụ
Thanh ghi đầu vào	Chốt setpoint, góc đo, gyro, ga, trạng thái arm khi có xung `sample_valid` (mẫu IMU mới)
MUX trục + bộ trừ	Chọn trục đang tính (roll, pitch hoặc yaw) và tính sai số e = setpoint − đo đạc
Trạng thái 3 trục	Thanh ghi nhớ tích phân A, sai số lần trước, đạo hàm đã lọc Df và đầu ra của từng trục
Tiền xử lý	Tính A + e, hiệu sai số ΔE và (ΔE − Df) bằng bộ cộng/trừ
Hằng số	Hệ số KP', KI', KD' mỗi trục, alpha bộ lọc, số bit dịch, các giới hạn — nối cứng vào mạch
Bộ nhân dùng chung	Linh kiện to nhất; lần lượt tính α·(ΔE − Df), KP'·e, KI'·A, KD'·Df cho cả 3 trục
Dịch phải + làm tròn	Đưa tích về đúng thang số nguyên, có làm tròn như mô hình
Σ, bão hoà, cắt, giới hạn	Cộng P + I + D; giới hạn ±250 và chống tràn tích phân; cắt về số nguyên; giới hạn ±250 (roll, pitch) hoặc ±80 (yaw)
Mixer X-frame	Trộn thành lệnh 4 động cơ, kẹp trong 1000–2000 µs; chưa arm hoặc ga ≤ 1000 thì cả 4 motor về 1000

4.3. Vì sao chỉ dùng một bộ nhân?

Drone chỉ cần tính PID **400 lần mỗi giây**, trong khi clock của chip chạy **10 triệu nhịp mỗi giây**. Thời gian rảnh rất dư, nên thay vì làm ba bộ PID riêng cho roll, pitch, yaw, mạch dùng **một bộ nhân, tính lần lượt từng trục** — giống một thu ngân phục vụ nhanh ba khách thay vì mở ba quầy. Cả ba trục tính xong trong 20 nhịp, tức **2 µs**, trong khi thời gian cho phép là **2500 µs**.



Hình 4. Lịch 20 chu kỳ clock cho một mẫu IMU. Mỗi trục đi qua 6 bước; ô đỏ là lúc bộ nhân cho kết quả dùng được.

4.4. Cổng vào/ra của chip

Cổng	Hướng	Ý nghĩa
`clk`, `rst_n`	vào	Clock; reset (mức thấp)
`sample_valid`	vào	Xung báo có mẫu IMU mới
`clear`	vào	Xoá trạng thái PID
`armed`, `throttle[10:0]`	vào	Trạng thái arm, mức ga (µs)
`sp_roll`, `sp_pitch`, `sp_yaw`	vào	Setpoint (độ, độ/s), số nguyên 22 bit có dấu
`roll`, `pitch`, `gz`	vào	Góc và tốc độ quay đo từ IMU, cùng định dạng
`m1..m4[10:0]`	ra	Lệnh 4 động cơ (µs, 1000–2000)
`mix_roll/pitch/yaw[9:0]`	ra	Đầu ra PID từng trục
`out_valid`, `busy`	ra	Báo có kết quả mới; báo đang tính

4.5. Độ rộng các thanh ghi chính (cấu hình A)

Tín hiệu	Số bit	Tín hiệu	Số bit
Đầu vào (góc, gyro, setpoint)	22	Sai số e	23
Tích phân A	27	Đạo hàm đã lọc Df	28
Tổng u' = P + I + D	28	Bộ nhân (toán hạng)	14 × 29

Các con số này được tính theo trường hợp xấu nhất (góc ±180°, gyro ±2000°/s) để mạch **không bao giờ tràn số**. Chúng nằm trong file `pid_params.vh` do Python tự sinh, nên mô hình và mạch luôn khớp nhau.

5. Bước 4 — Kiểm thử

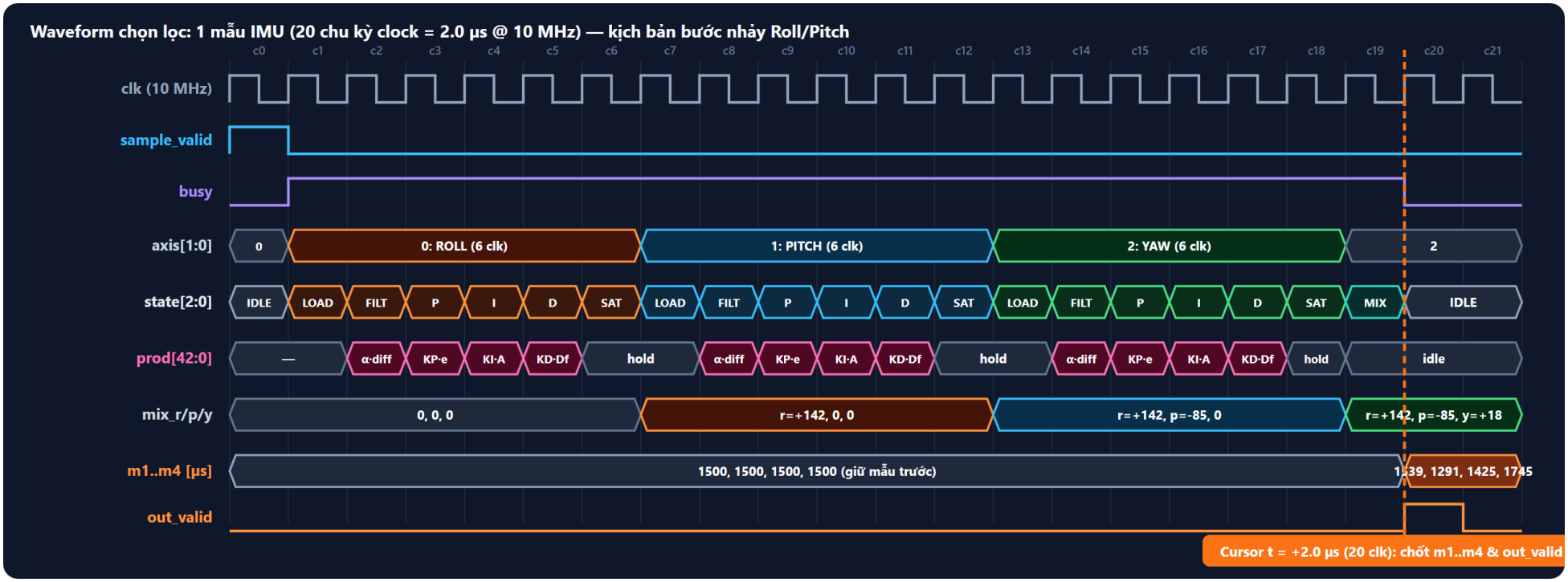
Chip đã sản xuất rồi thì không sửa được, nên phải chạy thử trên máy tính trước.

- **Testbench** (`tb_pid_ctrl.v`) đóng vai "người chấm bài": đưa dữ liệu vào mạch, đọc kết quả, so với đáp án.
- **Golden vectors** (`vectors.txt`) là bộ đề kèm đáp án: 16.000 mẫu, mỗi mẫu gồm đầu vào và đầu ra đúng do mô hình Python tính sẵn.
- **Bit-exact**: mạch phải cho ra đúng từng bit giống đáp án, không lệch dù chỉ 1.

> **Kết quả:** mô phỏng bằng Icarus Verilog 14 cho **PASS 16.000/16.000 mẫu** — mạch khớp từng bit với mô hình Python, mỗi mẫu tính xong trong đúng 20 chu kỳ clock, biên dịch với `-Wall` không có cảnh báo nào. Cả cấu hình A và B đều PASS. Để chắc testbench thật sự bắt được lỗi, nhóm đã thử sửa sai 1 đáp án trong bộ đề: testbench báo đúng `FAIL: 1/16000` và chỉ ra mẫu bị sai. Chạy lại bằng lệnh `sh ic/sim/run_iverilog.sh`.

6. Waveform: một mẫu IMU trong 20 chu kỳ clock

Hình dưới là dạng sóng mô phỏng của **một mẫu IMU** trong kịch bản đổi góc roll/pitch, phóng to vào đúng 20 nhịp clock (10 MHz, mỗi nhịp 0,1 µs). Đây là hình dùng ở vùng số 1 của Slide 8.



Hình 5. `pid_ctrl.v` xử lý một mẫu IMU: FSM đi qua LOAD → FILT → P → I → D → SAT cho từng trục rồi MIX. Con trỏ đặt tại +2,0 µs (chu kỳ thứ 20): `m1..m4` được chốt và `out_valid` lên 1 nhịp.

6.1. Đọc hình từ trên xuống

- **`sample_valid`** (c0): xung 1 nhịp báo có mẫu IMU mới. Mạch chốt toàn bộ đầu vào (setpoint, góc, gyro, ga, arm) vào thanh ghi.
- **`busy`** lên từ c1 đến hết c19: mạch đang tính. Trong lúc này đầu vào bên ngoài có đổi cũng không ảnh hưởng.
- **`axis`**: 6 nhịp cho roll, 6 nhịp cho pitch, 6 nhịp cho yaw — **một bộ nhân dùng lần lượt** cho cả ba trục.
- **`state`**: mỗi trục đi qua 6 bước. LOAD tính sai số e; FILT, P, I, D lấy kết quả bộ nhân; SAT bão hoà, chống tràn tích phân và ghi trạng thái.
- **`prod`**: bộ nhân cho lần lượt 4 tích α·(ΔE − Df), KP'·e, KI'·A, KD'·Df. Ô "hold" là lúc bộ nhân không cần dùng. Nhờ nạp toán hạng cho phép nhân kế tiếp ngay khi đang lấy kết quả phép trước, bộ nhân bận 4 trên 6 nhịp mỗi trục.
- **`mix_r/p/y`**: đầu ra từng trục được cập nhật ngay sau bước SAT của trục đó (r = +142, rồi p = −85, rồi y = +18).
- **`m1..m4`** giữ giá trị của mẫu trước suốt quá trình tính, chỉ đổi một lần ở bước MIX, nên 4 động cơ luôn nhận lệnh nhất quán.
- **Con trỏ +2,0 µs**: `out_valid` lên đúng 1 nhịp, 4 lệnh động cơ mới (1539, 1291, 1425, 1745 µs) sẵn sàng. Từ đây mạch nghỉ tới mẫu IMU sau — 2.498 µs còn lại của chu kỳ 2.500 µs.

> **Ý chính khi trình bày:** 20 nhịp = 1 nhịp chốt đầu vào + 3 trục × 6 nhịp + 1 nhịp mixer. Ở 10 MHz là 2,0 µs, tức chỉ 0,08% chu kỳ điều khiển. Testbench đo độ trễ của cả 16.000 mẫu và báo tối đa đúng 20 chu kỳ.

7. Đóng gói giao tiếp: `spi_pid_wrapper.v` (chống pad-limited)

7.1. Vấn đề: lõi PID có quá nhiều chân

Nếu đưa thẳng lõi `pid_ctrl` ra chân chip, mỗi bit dữ liệu là một chân (pad):

Nhóm tín hiệu	Phép tính	Số chân
6 đầu vào setpoint và đo đạc	6 × 22 bit	132
Ga (throttle)	11 bit	11
4 lệnh động cơ	4 × 11 bit	44
3 đầu ra PID từng trục	3 × 10 bit	30
Điều khiển (clk, rst_n, clear, sample_valid, armed, out_valid, busy…)	khoảng 6	6
Tổng		**≈ 223**

Mỗi pad trên chip là một ô lớn (thường cỡ 50–100 µm mỗi cạnh). Với hơn 200 pad, **vành pad bao quanh chip lớn hơn lõi logic rất nhiều lần**: diện tích chip bị quyết định bởi số chân chứ không phải bởi mạch. Đó gọi là **pad-limited**.

7.2. Giải pháp: gom hết vào 4 dây SPI

`ic/rtl/spi_pid_wrapper.v` bọc lõi PID trong một **SPI slave Mode 0** (CPOL = 0, CPHA = 0, bit cao gửi trước). ESP32 làm master, mỗi lần gửi một **khung 192 bit** (24 byte):

Chiều	Nội dung khung 192 bit

Chiều	Nội dung khung 192 bit
MOSI (ESP32 → chip)	`[191]` clear · `[190]` armed · `[189:179]` throttle · 6 trường 22 bit: sp_roll, sp_pitch, sp_yaw, roll, pitch, gz · 47 bit dự trữ (dành cho CRC)
MISO (chip → ESP32)	`[191]` busy · m1, m2, m3, m4 (11 bit mỗi cái) · mix_roll, mix_pitch, mix_yaw (10 bit mỗi cái) · phần còn lại là 0. Đây là **kết quả của mẫu trước**.

- **Chân của chip:** `clk`, `rst_n`, `spi_sclk`, `spi_cs_n`, `spi_mosi`, `spi_miso`, `irq_out_valid` — **7 chân** (cộng VDD, VSS). File RTL còn đưa ra chân `busy`, nhưng busy đã nằm ở bit 191 của MISO nên có thể bỏ chân này khi đóng gói.
- **Chống metastability:** SCLK, CS và MOSI đến từ ESP32 với clock khác, nên không dùng SCLK làm clock. SCLK và CS đi qua chuỗi 3 flip-flop trong miền `clk` (2 tầng đồng bộ + 1 tầng phát hiện cạnh), MOSI qua 2 flip-flop. Cả chip chỉ có một miền clock.
- **Chỉ nhận khung đủ:** khi CS lên, khung chỉ được đưa vào lõi nếu đã đếm đủ ít nhất 145 bit. Khung bị đứt giữa chừng không làm hỏng trạng thái PID. Bit `clear` = 1 thì khung đó chỉ xoá trạng thái, không tính mẫu.
- **Tốc độ SPI:** vì SCLK được lấy mẫu bằng `clk`, SCLK phải chậm hơn khoảng **f_clk / 4** (≈ 2,5 MHz với clk 10 MHz). Ví dụ SCLK 2 MHz: 192 bit mất 96 µs, cộng 2 µs tính là khoảng 98 µs — vẫn chưa tới 4% chu kỳ IMU 2.500 µs.

> **Kiểm thử:** testbench `tb/tb_spi_pid_wrapper.v` đóng vai ESP32, gửi 100 mẫu đầu của `vectors.txt` qua SPI, kiểm tra bit-exact cả chiều MOSI (giá trị đưa vào lõi) và MISO (kết quả đọc ở khung kế tiếp). Kết quả Icarus Verilog 12, SCLK 1 MHz: **PASS 100/100**, chạy đủ bộ đề cũng **PASS 16.000/16.000**; sửa sai 1 đáp án thì báo đúng FAIL. Lệnh chạy ghi ở đầu file testbench.

8. So sánh với các thiết kế đã công bố

Bảng dùng ở Slide 7. Hiện chỉ có baseline MCU của nhóm; thiết kế đã công bố sẽ được thêm khi đối chiếu xong DOI. Mỗi số đều kèm **bậc** (RTL sim, post-synthesis, post-layout, đo trên silicon) và **corner** (điều kiện điện áp, nhiệt độ). Ô của nhóm ghi "ước tính" vì chưa có báo cáo Genus.

Thiết kế & nguồn	Node / bậc / corner	Tần số	Diện tích	Công suất	Độ trễ mỗi vòng	FoM (nJ/loop)
Nhóm — cấu hình A (PID 3 trục + mixer, nhân 14×29)	PDK Cadence · post-synth (chờ) · TT 25°C / SS 125°C	10 MHz (20 clk/mẫu)	chờ Genus (ước tính 2,8k–3,5k GE)	chờ Genus (ước tính 0,25 mW)	2,0 µs	ước tính 0,50
Nhóm — cấu hình B (nhân 11×21)	PDK Cadence · post-synth (chờ) · TT 25°C / SS 125°C	10 MHz (20 clk/mẫu)	chờ Genus (ước tính 2,1k GE)	chờ Genus (ước tính 0,18 mW)	2,0 µs	ước tính 0,36
[1] Phần mềm trên STM32F103C8 (baseline của nhóm)	MCU 72 MHz, 3,3 V · datasheet DS5319, thời gian ước tính	72 MHz	— (chip MCU rời)	118,8 mW (36 mA × 3,3 V)	≈ 18,5 µs	≈ 2.198

8.1. FoM: thước đo đúng cho chip điều khiển

Chip mạng đo bằng Gbps; chip điều khiển bay chạy pin cần biết **tốn bao nhiêu năng lượng cho mỗi lần cập nhật 3 trục**:

`FoM = P × t_loop = P × (20 / f_clk)` [nJ/loop] — càng nhỏ càng tốt.

- Cấu hình A (ước tính): 0,25 mW × 2,0 µs = **0,50 nJ/loop**.
- STM32: 118,8 mW × 18,5 µs ≈ **2.198 nJ/loop**, tức gấp hơn 4.000 lần.
- FoM trên chỉ tính **lúc đang tính**. Năng lượng thật mỗi chu kỳ 2,5 ms còn cộng thêm điện rò lúc nghỉ: `E = P_active × 2 µs + P_leak × 2.498 µs`. Khi có báo cáo Genus, nhóm sẽ ghi cả hai con số.

8.2. Normalization: so sánh khi khác công nghệ

- **Diện tích → gate equivalent (GE):** chia diện tích cho diện tích một cổng NAND2 của chính thư viện đó. GE gần như không phụ thuộc node, nên so được chip 180 nm với chip 45 nm.
- **Quy đổi theo kích thước node (Dennard):** `Area_norm = Area × (L_target / L_ref)²`, `Power_norm = Power × (V_target / V_ref)² × (L_target / L_ref)`. Công thức này kém chính xác dưới khoảng 90 nm, nên chỉ dùng để so bậc độ lớn.
- **FPGA** không quy ra GE tin cậy được: giữ nguyên LUT/DSP và so bằng FoM năng lượng.
- Chỉ so **cùng chức năng**: PID 3 trục + mixer.

> **Trước khi nộp:** (1) điền số post-synthesis từ `reports/pid_ctrl_area.rpt` và `pid_ctrl_power.rpt` thay cho các ô "ước tính"; (2) chỉ thêm hàng thiết kế đã công bố khi có DOI và đã đối chiếu từng con số với bài báo gốc (hai trích dẫn cũ Lee 2021 và Hassan 2020 không tìm thấy nên đã gỡ); (3) số 18,5 µs của STM32 nên đo lại bằng bộ đếm `DWT->CYCCNT` trên board, còn 36 mA là số trong datasheet.

9. Các file trong thư mục `ic/`

File	Vai trò
`model/pid_model.py`	Hai bản PID: số thực (chép STM32) và số nguyên (giống mạch)
`model/analyze.py`	Thử các mức bit, chọn cấu hình, xuất báo cáo
`model/out/report.md`	Báo cáo kết quả chi tiết, dùng được cho slide và báo cáo thi
`model/export_rtl.py`	Sinh file tham số và bộ đề cho mạch (chọn cấu hình A hoặc B)
`rtl/pid_ctrl.v`	**Mạch PID** — sản phẩm chính
`rtl/pid_params.vh`	Số bit, hệ số, giới hạn — sinh tự động, không sửa tay
`rtl/spi_pid_wrapper.v`	Vỏ SPI 192 bit bọc lõi PID, giảm số chân từ ≈ 223 xuống 7
`tb/tb_pid_ctrl.v`, `tb/vectors.txt`	Người chấm bài và bộ đề
`tb/tb_spi_pid_wrapper.v`	Kiểm thử giao tiếp SPI với 100 mẫu đầu của bộ đề
`syn/syn_genus.tcl`	Script tổng hợp và xuất báo cáo diện tích, công suất, timing trên Cadence Genus
`sim/`	Lệnh chạy mô phỏng cho Icarus, Xcelium, ModelSim

10. Việc tiếp theo và điểm nhóm cần quyết định

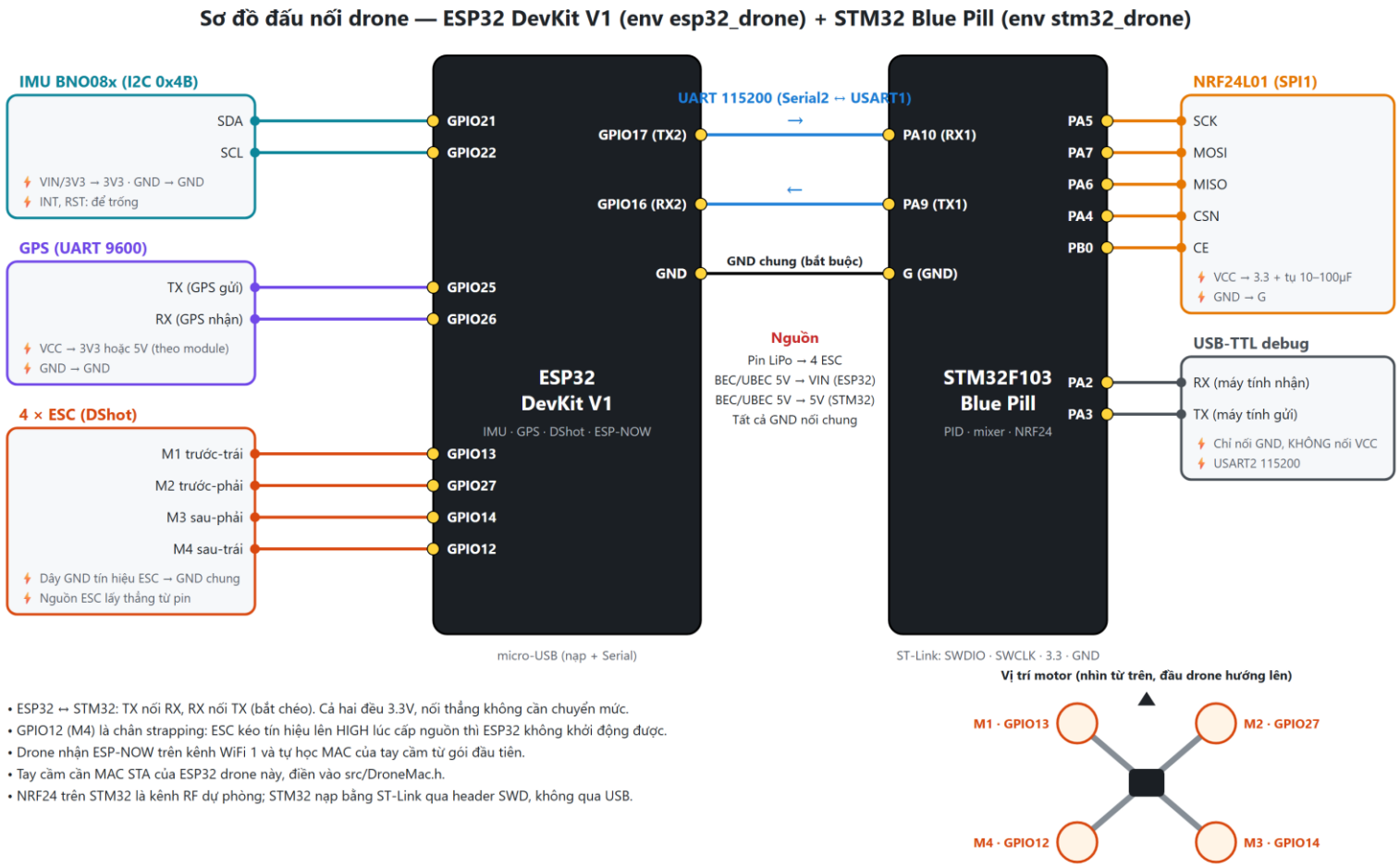
#	Việc	Ghi chú
1	Mô phỏng RTL	Đã xong: PASS 16.000/16.000 (Icarus Verilog)
2	Chọn cấu hình A hay B	Phân vân thì lấy A cho an toàn
3	Có đổi alpha thành 13/64 không	Tiết kiệm 7 bit; phải sửa cả STM32 và bay thử lại
4	Xác nhận tần số 400 Hz	PID chỉ tính khi có mẫu IMU mới; IMU đang cấu hình 2,5 ms
5	Có xoá trạng thái PID khi arm không	Code STM32 hiện giữ tích phân từ lần bay trước sang lần arm sau
6	Học Verilog cơ bản (1–2 buổi)	Đủ để đọc hiểu `pid_ctrl.v` và trả lời giám khảo
7	Buổi đào tạo Cadence	Chạy lại mô phỏng bằng Xcelium, rồi hỏi thẳng vào bước 5–7 (Genus, Innovus, kiểm tra cuối)

11. Thuật ngữ

Thuật ngữ	Nghĩa
Bit	Chữ số nhị phân (0 hoặc 1). Càng nhiều bit, số càng lớn/chính xác nhưng mạch càng to
Fixed-point (dấu phẩy tĩnh)	Lưu số thập phân dưới dạng số nguyên đã nhân với 2^n
Q.FE	Định dạng số nguyên có FE bit phần thập phân (ví dụ Q.10: đơn vị 1/1024)
Clock	Xung nhịp đều đặn; mỗi nhịp mạch tiến thêm một bước
Thanh ghi (register)	Ô nhớ nhỏ giữ một con số giữa các nhịp clock
MUX	Bộ chọn: chọn một trong nhiều đường dữ liệu đưa ra
FSM (máy trạng thái)	Bộ điều phối quyết định mỗi nhịp clock mạch làm việc gì
RTL / Verilog	Cách mô tả mạch bằng chữ; Verilog là ngôn ngữ dùng để viết
Testbench	Đoạn mã kiểm tra mạch: đưa dữ liệu vào, so kết quả với đáp án
Golden vectors	Bộ đề kèm đáp án đúng dùng để chấm mạch
Bit-exact	Kết quả khớp đúng từng bit với đáp án
Bão hoà / anti-windup	Giới hạn đầu ra; ngừng cộng tích phân khi đầu ra đã chạm giới hạn
Tổng hợp (synthesis)	Đổi Verilog thành mạch gồm các cổng logic có thật trong thư viện nhà máy
PDK	Bộ tài liệu của nhà máy: linh kiện có sẵn, luật vẽ, thông số
Layout / GDS	Bản vẽ vật lý của chip trên silicon; GDS là file gửi nhà máy

Phụ lục — Phần cứng nguyên mẫu với MCU

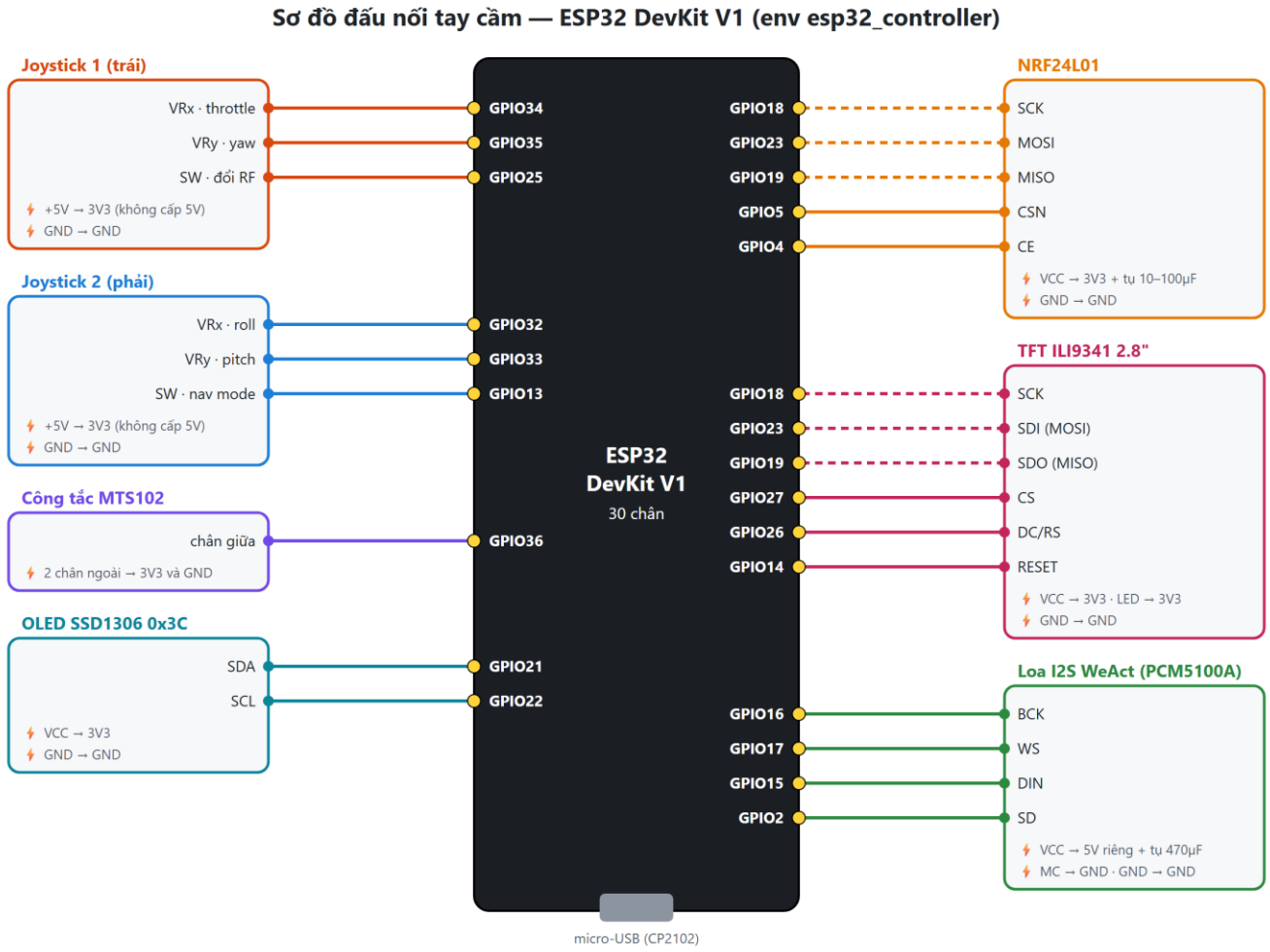
Các sơ đồ đấu nối của nguyên mẫu drone và tay cầm hiện tại, trên đó thuật toán PID đã được bay thử.



Hình A1. Drone: ESP32 (IMU, GPS, 4 ESC) và STM32 (PID, NRF24).



Hình A2. Tay cầm điều khiển trên WeAct ESP32-S3-A.



Hình A3. Tay cầm điều khiển trên ESP32 DevKit V1.